



Αναφορά ΜΕΛΟΥΣ ΟΜΑΔΑΣ 01 Προγραμματιστικού Project

Στοιχεία μέλους ομάδας: {Καταξενός Χρήστος, std170686@ac.eap.gr}

Τίτλος Project: Ανάπτυξη εφαρμογής για τη διαχείριση των ραντεβού μιας μικρής επιχείρησης

Τμήμα ΗΛΕ – 47

Ενότητα: ΠΛΗΠΡΟ

2025 – 2026

Περιεχόμενα

1. Ρόλος, ευθύνες και πλαίσιο συμμετοχής.....	2
2. Οργάνωση του έργου και αρχικός αρχιτεκτονικός σχεδιασμός	3
3. Υλοποίηση της βάσης δεδομένων και τεκμηρίωση της εργασίας	4
Δημιουργία εικονικών δεδομένων για τη διαδικασία ελέγχων (<i>seed_db.py</i>).....	4
4. Ανάπτυξη διεπαφής χρήστη και λειτουργική ολοκλήρωση.....	5
4.1 Το κεντρικό παράθυρο (<i>gui_main.py</i>).....	5
4.2 Το πάνελ ρυθμίσεων (<i>gui_settings.py</i>)	6
4.3 Ανάπτυξη μηχανισμού αντιγράφων ασφαλείας (<i>backup.py</i>).....	7
4.4 Μη γραμμική πορεία ανάπτυξης και συνεχείς βελτιώσεις.....	9
5. Προβλήματα στην τελική φάση και ανάληψη πρόσθετων αρμοδιοτήτων	9
6. Συνολική αποτίμηση της διαδικασίας και συμπεράσματα.....	10
7. Παραδοχές και περιορισμοί της εφαρμογής	11
1. Τοπική βάση δεδομένων και single-user λειτουργία.....	11
2. Ασφάλεια δεδομένων και GDPR	11
3. Υπηρεσία αποστολής email.....	11
4. Ανταπόκριση και περιορισμοί του γραφικού περιβάλλοντος	11
5. Αντίγραφα ασφαλείας και αποθήκευση δεδομένων	11
6. Back-office λειτουργία και ευέλικτη διαχείριση διαθεσιμότητας	12
7. Απουσία διασύνδεσης με εξωτερικά ημερολόγια.....	12
8. Επίλογος	13
9. Πηγές και βιβλιογραφία υλοποίησης	14

Στην παρούσα ατομική έκθεση περιγράφω τη δική μου συμμετοχή στο ομαδικό προγραμματιστικό έργο του μαθήματος ΠΛΗΠΡΟ, το οποίο είχε ως αντικείμενο την ανάπτυξη μιας εφαρμογής διαχείρισης ραντεβού για μικρή επιχείρηση. Στόχος μου δεν είναι μόνο να καταγράψω τα τεχνικά τμήματα που ανέλαβα, αλλά και να αποτυπώσω τη συνολική πορεία που ακολούθησα από τον Φεβρουάριο έως και την τελική παράδοση του έργου. Στο διάστημα αυτό χρειάστηκε να ασχοληθώ με ζητήματα οργάνωσης, αρχιτεκτονικού σχεδιασμού, υλοποίησης κώδικα, υποστήριξης της ομάδας και επίλυσης προβλημάτων που προέκυπταν στην πορεία.

Καθώς το έργο απαιτούσε γνώσεις που σε ορισμένα σημεία ξεπερνούσαν τις γνώσεις μου — ιδιαίτερα σε θέματα βάσεων δεδομένων, αρχιτεκτονικής και συντονισμού ομάδας — αξιοποίησα υποστηρικτικά ορισμένα εργαλεία τεχνητής νοημοσύνης αποκλειστικά για σκοπούς μελέτης και κατανόησης. Συγκεκριμένα, χρησιμοποίησα το Gemma 3 12B Q8_0 σε τοπικό περιβάλλον μέσω LM Studio για να κατανοήσω σε βάθος τον τρόπο λειτουργίας των βιβλιοθηκών που απαιτούνταν για την υλοποίηση της εφαρμογής. Παράλληλα, το Google NotebookLM με βοήθησε να καλύψω θεωρητικά κενά σε ζητήματα βάσεων δεδομένων και στον τρόπο υλοποίησης μέσω Python, καθώς δεν είχα προηγούμενη διδακτική εμπειρία στο αντικείμενο. Επιπλέον, το Microsoft Copilot λειτούργησε ως εργαλείο reasoning, βοηθώντας με στην ανάλυση προβλημάτων, στην οργάνωση της σκέψης μου και στη διαμόρφωση στρατηγικής ως προς τη δομή του κώδικα και τη συνεργασία της ομάδας. Είναι σημαντικό να τονιστεί ότι **δεν χρησιμοποιήθηκε πουθενά κώδικας παραγόμενος από AI**, ούτε ενσωματώθηκε αυτοματοποιημένο περιεχόμενο στο τελικό έργο, η χρήση των εργαλείων περιορίστηκε σε παραδείγματα, επεξηγήσεις και καθοδήγηση, ώστε να μπορέσω να ανταποκριθώ αποτελεσματικά στις τεχνικές και οργανωτικές απαιτήσεις του project.

1. Ρόλος, ευθύνες και πλαίσιο συμμετοχής

Από τις πρώτες εβδομάδες του έργου, και ιδιαίτερα μετά την πρώτη μας συνάντηση τον Φεβρουάριο, έγινε φανερό ότι η ομάδα χρειαζόταν ένα πιο σταθερό πλαίσιο συνεργασίας και μια κοινή λογική ανάπτυξης. Χωρίς να έχει οριστεί κάποιος τυπικός “αρχηγός”, ανέλαβα σταδιακά τον ρόλο του συντονιστή και του τεχνικά υπεύθυνου, κυρίως επειδή μπορούσα να παρακολουθώ τη συνολική εικόνα και να παρεμβαίνω όταν προέκυπταν τεχνικές δυσκολίες ή ασυνέχειες στη ροή του έργου.

Οι βασικές μου ευθύνες αφορούσαν τον αρχιτεκτονικό σχεδιασμό της εφαρμογής, την υλοποίηση κρίσιμων τμημάτων του κώδικα και τη διατήρηση μιας ενιαίας λογικής σε όλα τα επίπεδα — από τη βάση δεδομένων και τη λειτουργική λογική μέχρι τη διεπαφή χρήστη και την τελική ενσωμάτωση των modules. Παράλληλα, υποστήριζα τα υπόλοιπα μέλη της ομάδας όπου χρειαζόταν, είτε με τεχνικές διευκρινίσεις είτε με επίλυση προβλημάτων που εμπόδιζαν την πρόοδο των δικών τους τμημάτων.

Ως ομάδα αποφασίσαμε να υιοθετήσουμε μια full-stack προσέγγιση, όπου κάθε μέλος θα είχε την ευθύνη για ένα ολοκληρωμένο λειτουργικό κομμάτι — τόσο στο backend όσο και στο frontend. Η επιλογή αυτή δεν ήταν τυχαία, προέκυψε μέσα από συζήτηση και από την ανάγκη να μειώσουμε τις εξαρτήσεις μεταξύ των υποομάδων. Μέσα σε αυτό το πλαίσιο, η δική μου συμβολή απλώθηκε φυσικά σε περισσότερα επίπεδα, καθώς έπρεπε να διασφαλίσω ότι τα επιμέρους κομμάτια που ανέπτυξε ο καθένας θα μπορούσαν να ενσωματωθούν ομαλά σε ένα συνεκτικό σύνολο.

Καθ’ όλη τη διάρκεια του project παρακολουθούσα την πρόοδο των επιμέρους τμημάτων, συντονίζα τις τεχνικές αποφάσεις και βοηθούσα στη διατήρηση μιας κοινής κατεύθυνσης. Η συνεργασία με τα υπόλοιπα μέλη —τον Δημήτρη Ασπρίδη, τον Πάνο Βαρθαλίτη, την Καλλιόπη Καναβού και τον Γιώργο Κονδύλη— ήταν συνεχής και ουσιαστική, καθώς συζητούσαμε συχνά θέματα

λογικής, δοκιμών και σχεδιασμού, ενώ ο Κονδύλης Γιώργος συνέβαλε σημαντικά με τις εύστοχες παρατηρήσεις του καθ' όλη τη διάρκεια του project και με τις ιδέες που προέκυπταν μέσα από τα tests που εκτελούσε για τη λειτουργία της εφαρμογής, ώστε το τελικό αποτέλεσμα να παραμείνει συνεπές και λειτουργικό.

2. Οργάνωση του έργου και αρχικός αρχιτεκτονικός σχεδιασμός

Στην αρχική φάση του project, ακολουθήσαμε το κλασικό μοντέλο οργάνωσης με διαχωρισμό σε frontend, backend και testing. Μέσα σε αυτό το πλαίσιο είχα αναλάβει κυρίως το frontend, καθώς είχα ήδη σχετική άνεση από προσωπικά DIY projects και πίστευα ότι μπορούσα να συμβάλω ουσιαστικά εκεί. Θεωρητικά, η προσέγγιση αυτή έμοιαζε λογική, όμως στην πράξη αποδείχθηκε ότι δεν ταίριαζε στον τρόπο που λειτουργούσε η ομάδα μας ούτε στο επίπεδο εμπειρίας που είχαμε όλοι εκείνη την περίοδο.

Το βασικό πρόβλημα δεν ήταν η διάθεση — αυτή υπήρχε. Το ζήτημα ήταν ότι ο αυστηρός διαχωρισμός ρόλων δημιουργούσε εξαρτήσεις: για να προχωρήσει ένα κομμάτι, έπρεπε κάποιος άλλος να έχει ήδη ολοκληρωθεί ή τουλάχιστον να έχει ξεκαθαρίσει. Αυτό επιβράδυνε τη ροή και, κυρίως, δεν βοηθούσε τα μέλη να κατανοήσουν πώς “δένει” η εφαρμογή από τη λογική μέχρι τη διεπαφή. Εκεί συνειδητοποίησα ότι, αν συνεχίζαμε έτσι, θα είχαμε περισσότερη αναμονή παρά δημιουργία.

Για αυτό πρότεινα μια διαφορετική προσέγγιση: κάθε μέλος να αναλάβει ένα ολοκληρωμένο λειτουργικό κομμάτι της εφαρμογής και να το υλοποιήσει full-stack, τόσο στο επίπεδο της λογικής όσο και στο επίπεδο της διεπαφής. Η σκέψη πίσω από αυτή την πρόταση ήταν διπλή. Από τη μία, ήθελα ο καθένας να αποκτήσει ουσιαστική κατανόηση του προγραμματισμού μέσα από ένα πλήρες κομμάτι και όχι μέσα από ένα στενό τεχνικό υποσύνολο. Από την άλλη, στόχος ήταν να μειωθούν οι συνεχείς εξαρτήσεις μεταξύ των υποομάδων, ώστε κανείς να μη χρειάζεται να περιμένει τους άλλους για να συνεχίσει.

Μέσα σε αυτό το νέο πλαίσιο, ένα από τα πρώτα πρακτικά βήματα που ανέλαβα ήταν η διαμόρφωση του βασικού σκελετού της εφαρμογής στον φάκελο /src. Ήθελα να υπάρχει από νωρίς μια κοινή λογική για το πού ανήκει κάθε μέρος του κώδικα και πώς συνδέονται μεταξύ τους τα modules. Δεν ήταν απλώς θέμα φακέλων ή ονοματοδοσίας, ήταν ένας τρόπος να αποκτήσει η ομάδα ένα σταθερό σημείο αναφοράς, ώστε ακόμη κι αν ο καθένας δούλευε το δικό του κομμάτι με τον δικό του ρυθμό, να μην προκύψουν ασύνδετες λύσεις. Με πιο απλά λόγια, προσπάθησα να εισάγω μια αρχιτεκτονική πειθαρχία χωρίς να περιορίσω την ευελιξία που χρειαζόμασταν.

Κοιτώντας το έργο εκ των υστέρων, θεωρώ ότι αυτή η μεθοδολογική αλλαγή ήταν μία από τις σημαντικότερες αποφάσεις της αρχικής φάσης. Παρότι δεν έλυσε όλα τα προβλήματα, βελτίωσε αισθητά τον ρυθμό της ομάδας, ενίσχυσε την αυτονομία των μελών και έκανε πιο σαφή την κατανομή της εργασίας. Παράλληλα, μου επέτρεψε να συμβάλω όχι μόνο ως προγραμματιστής σε επιμέρους αρχεία, αλλά και στη διαμόρφωση μιας λειτουργικής και καλά οργανωμένης αρχιτεκτονικής για το σύνολο του έργου.

3. Υλοποίηση της βάσης δεδομένων και τεκμηρίωση της εργασίας

Για την υλοποίηση μελέτησα το επίσημο documentation της Python για το sqlite3, καθώς και πρόσθετες τεχνικές πηγές. Η ανάπτυξη του αρχείου αρχικού κορμού για το *database.py* ήταν από τα πιο απαιτητικά μέρη του έργου και χρειάστηκε περίπου 25–35 ώρες σε συμπυκνωμένο χρονικό διάστημα, κυρίως σε ένα ιδιαίτερα εντατικό Σαββατοκύριακο, ώστε τα υπόλοιπα μέλη της ομάδας να μπορέσουν να ξεκινήσουν έγκαιρα τα δικά τους modules.

Σε αυτό το στάδιο αξιοποίησα υποστηρικτικά και το Google NotebookLM, αποκλειστικά για σκοπούς κατανόησης. Με βοήθησε να κατανοήσω τον τρόπο λειτουργίας των βάσεων δεδομένων, τις βασικές αρχές σχεδιασμού τους και ταυτόχρονα πώς μεταφράζονται αυτές οι έννοιες σε πρακτική υλοποίηση μέσω Python. Μέσα από παραδείγματα και επεξηγήσεις μπόρεσα να διαμορφώσω με μεγαλύτερη σιγουριά τα απαραίτητα queries και τα functions του *database.py*. Είναι σημαντικό να τονιστεί ότι η χρήση του περιορίστηκε αποκλειστικά στη θεωρητική κατανόηση και όχι στην παραγωγή κώδικα, ο οποίος υλοποιήθηκε εξ ολοκλήρου από εμένα στην πρώτη φάση υλοποίησης.

Η διαδικασία περιλάμβανε:

- ορισμό πινάκων και σχέσεων μεταξύ των οντοτήτων,
- υλοποίηση CRUD λειτουργιών για όλες τις βασικές οντότητες,
- ενσωμάτωση μηχανισμών ακεραιότητας μέσω foreign keys και constraints,
- προσαρμογή του μοντέλου ώστε να υποστηρίζει τις λειτουργίες της εφαρμογής σε πραγματικές συνθήκες χρήσης.

Ο βασικός κορμός του *database.py* υλοποιήθηκε από εμένα, στηριζόμενος τόσο στα UML/ER models όσο και στην αρχική βάση που είχε δημιουργήσει η Καλλιόπη. Ωστόσο, η βάση εξελίχθηκε μέσα από τη συνεργασία όλης της ομάδας. Κατά τη διάρκεια του development:

- ο Δημήτρης Ασπρίδης πρόσθεσε και τροποποίησε functions που σχετίζονταν με τις ανάγκες των δικών του modules,
- ο Πάνος Βαρθαλίτης προχώρησε σε στοχευμένες παρεμβάσεις ώστε η βάση να συνεργάζεται σωστά με τα search functions και τις λειτουργίες export που ανέπτυξε.

Οι παρεμβάσεις αυτές δεν ήταν απλές διορθώσεις, αλλά ουσιαστική προσαρμογή της βάσης στις πραγματικές ανάγκες της εφαρμογής. Η συνεργασία μας βοήθησε να εντοπίσουμε περιπτώσεις που δεν ήταν εμφανείς στο αρχικό θεωρητικό μοντέλο.

Καθώς η εφαρμογή άρχισε να δοκιμάζεται σε πιο ρεαλιστικά σενάρια χρήσης, ο κορμός του *database.py* χρειάστηκε αρκετές αλλαγές. Η διαδικασία αυτή ήταν ιδιαίτερα σημαντική, γιατί ανέδειξε ότι η σωστή λειτουργία μιας βάσης δεδομένων δεν κρίνεται μόνο από τον θεωρητικό σχεδιασμό, αλλά από το κατά πόσο αντέχει στις πραγματικές συνθήκες χρήσης.

Ένα χαρακτηριστικό παράδειγμα ήταν η λογική διαχείρισης των συγκρούσεων ραντεβού. Η αρχική υλοποίηση δεν κάλυπτε όλες τις περιπτώσεις επικαλύψεων, οπότε χρειάστηκε να επανεξετάσω τον αλγόριθμο και να τον βελτιώσω, ώστε το σύστημα να αναγνωρίζει με μεγαλύτερη ακρίβεια πότε δύο ραντεβού παραβιάζουν τους χρονικούς περιορισμούς. Μέσα από αυτές τις παρεμβάσεις, η βάση έγινε πιο σταθερή και αξιόπιστη στην πράξη, και όχι μόνο σε επίπεδο σχεδίασης.

Δημιουργία εικονικών δεδομένων για τη διαδικασία ελέγχων (*seed_db.py*)

Κατά τη φάση των δοκιμών διαπίστωσα ότι η εφαρμογή χρειαζόταν μεγαλύτερο όγκο δεδομένων, ώστε να μπορούν να ελεγχθούν σωστά λειτουργίες όπως η αναζήτηση, το ημερολόγιο, τα

στατιστικά και η ανίχνευση επικαλύψεων. Επειδή η χειροκίνητη εισαγωγή εκατοντάδων εγγραφών θα ήταν ιδιαίτερα χρονοβόρα, ανέπτυξα το `seed_db.py`, ένα αυτοματοποιημένο script δημιουργίας δεδομένων για τη βάση `randeboo.db`. Για την παραγωγή mockup data χρησιμοποίησα τη βιβλιοθήκη `Faker` με ελληνικό locale, ενώ παράλληλα φρόντισα τα παραγόμενα δεδομένα να είναι συμβατά με τη λογική της εφαρμογής, αποφεύγοντας μη εργάσιμες ημέρες, αργίες και προσαρμοσμένο πρόγραμμα επιχείρησης μέσω του `get_business_settings`.

Το script σχεδιάστηκε ώστε να μπορεί είτε να δημιουργεί νέους πελάτες, υπαλλήλους και ραντεβού είτε να αξιοποιεί υπάρχουσες εγγραφές της βάσης, μέσω ενός απλού διαδραστικού μενού γραμμής εντολών. Η χρήση του αποδείχθηκε ιδιαίτερα χρήσιμη στη διαδικασία ελέγχου, γιατί μου επέτρεψε να προσομοιώσω γρήγορα σενάρια πραγματικής λειτουργίας και να δοκιμάσω την απόκριση του συστήματος με μεγάλο όγκο δεδομένων, χωρίς να απαιτείται συνεχής χειροκίνητη καταχώριση.

4. Ανάπτυξη διεπαφής χρήστη και λειτουργική ολοκλήρωση

Παράλληλα με την ανάπτυξη του `database.py`, ανέλαβα και σημαντικό μέρος της διεπαφής χρήστη της εφαρμογής. Για μένα, το UI δεν ήταν απλώς ένα σύνολο από οθόνες και κουμπιά, ήταν ο τρόπος με τον οποίο η εφαρμογή θα αποκτούσε συνοχή, ρυθμό και μια ολοκληρωμένη ταυτότητα ως προϊόν. Από νωρίς ήθελα να αποφύγουμε μια εμπειρία χρήσης που θα θύμιζε ασύνδετα παράθυρα, καθώς αυτό θα έκανε την εφαρμογή πιο κουραστική και λιγότερο επαγγελματική.

4.1 Το κεντρικό παράθυρο (`gui_main.py`)

Με αυτή τη λογική υλοποίησα το κεντρικό παράθυρο της εφαρμογής (`gui_main.py`), το οποίο λειτουργεί ως dashboard και ως σημείο φόρτωσης των επιμέρους ενοτήτων. Η προσέγγιση αυτή παραπέμπει σε λογική Single Page Architecture, καθώς όλες οι οθόνες φορτώνονται δυναμικά μέσα στο ίδιο κεντρικό container, χωρίς να ανοίγουν νέα παράθυρα. Για να υποστηριχθεί αυτή η συμπεριφορά, υλοποίησα έναν απλό αλλά αποτελεσματικό μηχανισμό routing.

Ο μηχανισμός αποτελείται από τρία βασικά στοιχεία:

- **Routing Map:** ένα dictionary που αντιστοιχίζει τα ονόματα των views με τις αντίστοιχες μεθόδους σχεδίασης.
- **Content Frame:** ένα κοινό frame που λειτουργεί ως “καμβάς” για όλες τις σελίδες.
- **Router Function:** μια κεντρική μέθοδος (`open_view`) που καθαρίζει το περιεχόμενο και φορτώνει τη νέα σελίδα.

Παραθέτω ένα μικρό ενδεικτικό απόσπασμα κώδικα που δείχνει τον τρόπο με τον οποίο γίνεται η εναλλαγή των panels:

```
def open_view(self, name: str) -> None:
    """Κεντρικός router για την εναλλαγή των panels (SPA)."""
    if name not in self.views:
        logging.warning(f"Unknown view: {name}")
        return

    self._clear_content()
    self.views[name]() # Κλήση της αντίστοιχης συνάρτησης σχεδίασης

def _clear_content(self):
    """Αδειάζει το κεντρικό frame πριν φορτώσει νέα σελίδα."""
    for widget in self.content_frame.wininfo_children():
        widget.destroy()
```

Με αυτόν τον τρόπο, το `gui_main.py` λειτουργεί ως ενιαίο σημείο πλοήγησης, επιτρέποντας στην εφαρμογή να συμπεριφέρεται σαν Single Page Application, με ομαλή εναλλαγή περιεχομένου και συνεκτική εμπειρία χρήσης.

Κατά τη διάρκεια της ανάπτυξης του `gui_main.py`, καθώς διαμόρφωνα τη βασική δομή και τη συνολική εμπειρία πλοήγησης, άρχισε να διαμορφώνεται σταδιακά και η αισθητική κατεύθυνση που ονόμασα Aegean Theme. Η ανάγκη για ένα πιο καθαρό και σύγχρονο περιβάλλον χρήσης προέκυψε φυσικά μέσα από τη διαδικασία σχεδίασης του κεντρικού παραθύρου, καθώς δεν ήθελα η εφαρμογή να θυμίζει ένα τυπικό, παλιού τύπου Tkinter πρόγραμμα με προεπιλεγμένα στοιχεία του λειτουργικού συστήματος.

Το Aegean Theme στηρίχθηκε σε τρεις βασικούς άξονες:

- αξιοπιστία,
- καθαρότητα πληροφορίας,
- οπτική απλότητα.

Για τον σκοπό αυτό χρησιμοποίησα το Deep Greek Blue (#0A3D62) στο sidebar και στους τίτλους, ώστε να δίνει σταθερότητα και σαφή οριοθέτηση στον χώρο πλοήγησης. Το Aegean Blue (#1E90FF) λειτούργησε ως χρώμα έμφασης στα κύρια κουμπιά και στα ενεργά στοιχεία, ενώ οι ανοιχτές αποχρώσεις όπως το Ice White και το Soft Grey (#F7F9FC, #FFFFFF) χρησιμοποιήθηκαν στο φόντο και στα panels, για να διατηρείται η ευαναγνωσιμότητα και η ξεκούραστη παρουσίαση του περιεχομένου. Επιπλέον, στοχευμένα χρώματα όπως πράσινο, κόκκινο και πορτοκαλί χρησιμοποιήθηκαν για άμεση οπτική αναγνώριση ενεργειών όπως αποθήκευση, διαγραφή ή ακύρωση.

Επειδή το Aegean Theme εξελίχθηκε σε βασικό στοιχείο της αισθητικής της εφαρμογής, το κατέγραψα και ως palette στο Adobe Color, ώστε να υπάρχει διαθέσιμο και για μελλοντικά projects.

Αυτή η αισθητική βάση που δημιουργήθηκε στο `gui_main.py` αποτέλεσε το σημείο αναφοράς για όλες τις υπόλοιπες οθόνες της εφαρμογής.

4.2 Το πάνελ ρυθμίσεων (`gui_settings.py`)

Μετά την ολοκλήρωση του βασικού κορμού της διεπαφής στο `gui_main.py`, προχώρησα στην ανάπτυξη του πάνελ ρυθμίσεων (`gui_settings.py`), το οποίο αποτέλεσε ένα από τα πιο σύνθετα και απαιτητικά τμήματα της εφαρμογής. Στόχος μου ήταν να δημιουργήσω ένα κεντρικό σημείο παραμετροποίησης που να συνδυάζει λειτουργικότητα, καθαρή οργάνωση και συνέπεια με την αισθητική κατεύθυνση που είχε ήδη διαμορφωθεί στο Aegean Theme.

Η υλοποίηση του `gui_settings.py` χρειάστηκε περίπου 25 ώρες, καθώς συγκέντρωνε ρυθμίσεις που επηρέαζαν άμεσα τη συμπεριφορά της εφαρμογής. Πέρα από τις βασικές επιλογές εμφάνισης και λειτουργίας, ανέπτυξα επιμέρους ενότητες που συνδέονται με κρίσιμες λειτουργίες:

- Ρύθμιση αποστολής email, με δυνατότητα προσχεδίου σύνταξης και χρήση placeholder functions.
- Καθορισμός ωραρίου λειτουργίας της επιχείρησης, πλήρως συνδεδεμένος με τη λογική των ραντεβού.
- Δυνατότητα αλλαγής κωδικού χρήστη, ως πρόσθετο μέτρο ασφάλειας.
- Ρύθμιση επισύναψης αρχείου .ics, η οποία δεν δημιουργεί η ίδια το αρχείο, αλλά αποθηκεύει την επιλογή στη βάση ώστε το `email_service.py` να αποφασίζει αν θα το επισυνάψει.

Ένα χαρακτηριστικό παράδειγμα της πολυπλοκότητας του πάνελ είναι ο τρόπος με τον οποίο συγκεντρώνονται και αποθηκεύονται όλες οι ρυθμίσεις. Η διαδικασία περιλαμβάνει ανάγνωση τιμών από widgets, μετατροπή σύνθετων δομών σε JSON και ενημέρωση της SQLite βάσης. Ενδεικτικά:

```
updated_settings_copy = self.settings.copy()

for key, widget in self.ents.items():

    updated_settings_copy[key] = widget.get()

updated_settings_copy.update({

    "business_description": self.text_description.get("1.0", "end-1c"),

    "weekly_schedule": json.dumps(new_schedule_state),

    "exception_days": json.dumps(self.temporary_settings_buffer),

    "email_body_template": self.text_email_template.get("1.0", "end-1c"),

    "add_to_calendar": "1" if self.var_ics_ref.get() else "0",

    "email_password": self.entry_email_pass.get()

})
```

Η αποθήκευση αυτή δεν είναι απλώς μια “συλλογή τιμών”, αλλά μια διαδικασία που συνδέει το UI με τη βάση δεδομένων και, τελικά, με τη λειτουργική λογική της εφαρμογής. Με αυτόν τον τρόπο, το `gui_settings.py` συνέβαλε καθοριστικά στη συνοχή και στην επαγγελματική αίσθηση του τελικού προϊόντος.

4.3 Ανάπτυξη μηχανισμού αντιγράφων ασφαλείας (*backup.py*)

Κατά την ανάπτυξη της εφαρμογής διαπίστωσα ότι, παρότι δεν υπήρχε η απαίτηση για μηχανισμό αντιγράφων ασφαλείας, μια τέτοια δυνατότητα ήταν ουσιαστική για τη μακροχρόνια αξιοπιστία του συστήματος. Η βάση δεδομένων περιέχει κρίσιμες πληροφορίες — πελάτες, ραντεβού, ωράρια, ρυθμίσεις — και η απώλειά τους θα μπορούσε να καταστήσει την εφαρμογή μη λειτουργική. Για τον λόγο αυτό ανέπτυξα το αρχείο `backup.py`, το οποίο προσθέτει έναν αυτόματο και ασφαλή μηχανισμό δημιουργίας και επαναφοράς αντιγράφων ασφαλείας.

Ο μηχανισμός δημιουργεί τοπικά αντίγραφα της SQLite βάσης, χρησιμοποιώντας timestamp για μοναδική ονομασία, και διατηρεί μόνο τα πιο πρόσφατα backups ώστε να μην συσσωρεύονται περιττά αρχεία. Παράλληλα, σε περίπτωση σφάλματος ή αλλοίωσης δεδομένων, ο χρήστης μπορεί να επαναφέρει άμεσα ένα προηγούμενο αντίγραφο. Η υλοποίηση χρειάστηκε περίπου 6 ώρες, συμπεριλαμβανομένης της μελέτης τεχνικών πηγών για ασφαλή χειρισμό αρχείων και logging.

Ενδεικτικά, η βασική λειτουργία δημιουργίας αντιγράφου ασφαλείας υλοποιήθηκε ως εξής:

```
def create_backup(silent: bool = True) -> bool:

    try:

        if not os.path.exists(DB_PATH):

            return False

        if not os.path.exists(BACKUP_DIR):

            os.makedirs(BACKUP_DIR, exist_ok=True)

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

        dest_path = os.path.join(BACKUP_DIR, f"backup_{timestamp}.db")
```

```
shutil.copy2(DB_PATH, dest_path)

cleanup_old_backups()
```

Στο backup.py χρησιμοποιήθηκε η συνάρτηση shutil.copy2 αντί της shutil.copy για έναν συγκεκριμένο λειτουργικό λόγο: η copy2, εκτός από τα δεδομένα του αρχείου και τα βασικά δικαιώματα πρόσβασης, επιχειρεί να διατηρήσει και τα διαθέσιμα μεταδεδομένα(metadata) του αρχείου, όπως τους χρόνους πρόσβασης και τελευταίας τροποποίησης. Αυτό έχει πρακτική σημασία σε ένα σύστημα αντιγράφων ασφαλείας, γιατί το αντίγραφο παραμένει πιο πιστό ως προς την κατάσταση του αρχικού αρχείου και δεν αντιμετωπίζεται απλώς ως ένα νέο αρχείο που δημιουργήθηκε τη στιγμή της αντιγραφής.

Η επιλογή αυτή συνδέεται και με τη λειτουργία cleanup_old_backups, καθώς σε έναν μηχανισμό που διατηρεί μόνο τα πιο πρόσφατα αντίγραφα είναι χρήσιμο τα αρχεία να κουβαλούν όσο το δυνατόν περισσότερη πληροφορία για την πραγματική χρονική τους σχέση με το αρχικό αρχείο. Παρότι η ονομασία με timestamp καλύπτει ήδη την αναγνώριση της σειράς δημιουργίας των backup, η διατήρηση metadata ενισχύει τη συνέπεια και την αξιοπιστία της διαδικασίας αποθήκευσης και επαναφοράς.

Το backup.py ενίσχυσε σημαντικά την αξιοπιστία της εφαρμογής, προσφέροντας ένα επίπεδο ασφάλειας που δεν υπήρχε στα αρχικά ζητούμενα, αλλά θεωρώ απαραίτητο για οποιοδήποτε σύστημα που διαχειρίζεται επιχειρησιακά δεδομένα. Η προσθήκη του δείχνει πώς η εφαρμογή μπορεί να εξελιχθεί πέρα από τις βασικές απαιτήσεις όταν εντοπίζονται πραγματικές ανάγκες που επηρεάζουν τη σταθερότητα και τη βιωσιμότητά της.

Στην τελική φάση της υλοποίησης χρειάστηκε επίσης να προσαρμόσω τα αρχεία backup.py και database.py ώστε να λειτουργούν σωστά και στην packaged εκτέλεση μέσω PyInstaller. Συγκεκριμένα, προστέθηκε έλεγχος του sys.frozen, ώστε όταν η εφαρμογή εκτελείται ως compiled executable να χρησιμοποιείται ως σημείο αναφοράς ο φάκελος του sys.executable, ενώ κατά το κανονικό development διατηρείται η λογική εντοπισμού του έργου μέσω του __file__. Η προσαρμογή αυτή ήταν απαραίτητη, γιατί σε περιβάλλον PyInstaller οι σχετικές διαδρομές μπορεί να οδηγούν σε προσωρινούς φακέλους αποσυμπίεσης και όχι στον πραγματικό φάκελο χρήσης του χρήστη. Αν παρέμενε η αρχική λογική, η βάση data/randeboo.db και τα αντίγραφα ασφαλείας θα μπορούσαν να αποθηκεύονται σε προσωρινή τοποθεσία, με αποτέλεσμα απώλεια δεδομένων μετά το κλείσιμο της εφαρμογής. Με τη λύση αυτή εξασφάλισα ότι τόσο η βάση όσο και ο φάκελος backups δημιουργούνται σταθερά δίπλα στο .exe του χρήστη.

Παράλληλα, χρειάστηκε να δημιουργήσω και το αρχείο RandeBoo.spec, δηλαδή το αρχείο ρυθμίσεων με το οποίο το PyInstaller χτίζει το τελικό εκτελέσιμο. Εκεί όρισα τη συμπερίληψη των στατικών αρχείων της εφαρμογής μέσω της παραμέτρου datas, ώστε εικόνες και εικονίδια όπως τα randeboo.ico και randeboo.png να βρίσκονται διαθέσιμα και μετά το build. Χωρίς αυτή τη ρύθμιση η εφαρμογή δεν εντόπιζε σωστά τους οπτικούς πόρους της κατά την εκκίνηση. Επιπλέον, αντιμετώπισα το ζήτημα των dynamic imports της βιβλιοθήκης holidays, χρησιμοποιώντας τη συνάρτηση collect_submodules, επειδή το PyInstaller δεν μπορεί πάντα να ανιχνεύσει αυτόματα υπομονάδες που φορτώνονται δυναμικά κατά το runtime. Τέλος, ρύθμισα το εκτελέσιμο ως καθαρά γραφική εφαρμογή με console=False, ώστε να μην εμφανίζεται παράθυρο τερματικού πίσω από το GUI, και όρισα το εικονίδιο του .exe ώστε η τελική διανομή να είναι πιο συνεπής και πιο κοντά σε πραγματικό επιτραπέζιο λογισμικό. Οι παρεμβάσεις αυτές δείχνουν ότι μέρος της συμβολής μου δεν περιορίστηκε στη συγγραφή του πηγαίου κώδικα, αλλά επεκτάθηκε και στη διαδικασία μετατροπής της εφαρμογής σε σταθερό, διανεμήσιμο προϊόν.

4.4 Μη γραμμική πορεία ανάπτυξης και συνεχείς βελτιώσεις

Η ανάπτυξη της διεπαφής χρήστη δεν ήταν μια γραμμική διαδικασία. Από τον Φεβρουάριο μέχρι την τελική παράδοση υπήρξαν περίοδοι προετοιμασίας, δοκιμών και ανασχεδιασμών, καθώς η εφαρμογή εξελισσόταν και προέκυπταν νέες ανάγκες. Πολλά τμήματα του UI χρειάστηκε να προσαρμοστούν ώστε να δένουν καλύτερα με τη συνολική αρχιτεκτονική και τη λειτουργική λογική της εφαρμογής, ενώ σε ορισμένες φάσεις η ένταση της εργασίας ήταν σημαντικά αυξημένη για να καλυφθούν τεχνικές εκκρεμότητες ή καθυστερήσεις σε άλλα μέρη του project.

Για αυτό θεωρώ ότι η συμβολή μου στη διεπαφή χρήστη και στις υποστηρικτικές λειτουργίες δεν αποτέλεσε ένα μεμονωμένο τεχνικό task, αλλά μια συνεχή και εξελισσόμενη διαδικασία που συνόδεψε όλη την ανάπτυξη της εφαρμογής και συνέβαλε ουσιαστικά στη συνοχή και την ωριμότητα του τελικού αποτελέσματος.

5. Προβλήματα στην τελική φάση και ανάληψη πρόσθετων αρμοδιοτήτων

Καθώς το project πλησίαζε στην ολοκλήρωσή του, έγινε εμφανές ότι ορισμένα λειτουργικά τμήματα δεν είχαν προχωρήσει στον βαθμό που απαιτούσε η συνολική ωριμότητα της εφαρμογής. Σε αυτό το στάδιο δεν υπήρχε πλέον η δυνατότητα να παραμείνουν ανοιχτά ζητήματα για “μετά”. Χρειάστηκε να ληφθούν γρήγορες αποφάσεις, ώστε η εφαρμογή να παραδοθεί με όσο το δυνατόν μεγαλύτερη λειτουργική πληρότητα.

Μέσα σε αυτή τη συνθήκη ανέλαβα επιπλέον αρμοδιότητες πέρα από τα κομμάτια που είχα ήδη υλοποιήσει. Ο ρόλος μου επεκτάθηκε τόσο τεχνικά όσο και οργανωτικά, με στόχο να καλυφθούν κενά που παρέμεναν ανοιχτά και να διασφαλιστεί ότι το τελικό αποτέλεσμα θα ήταν συνεκτικό και λειτουργικό.

Ένα χαρακτηριστικό παράδειγμα ήταν η υλοποίηση του module διαχείρισης προσωπικού (*gui_employees.py*). Για να επιταχύνω την ανάπτυξη και να αποφύγω την εκκίνηση από το μηδέν, βασίστηκα στη λογική του ήδη υπάρχοντος και πλήρως λειτουργικού module πελατών (*gui_customers.py*) και την προσαρμοσα στις ανάγκες της νέας ενότητας. Η επιλογή αυτή δεν ήταν απλή αντιγραφή, αλλά μια συνειδητή στρατηγική επαναχρησιμοποίησης δοκιμασμένης λογικής, ώστε να καλυφθεί γρήγορα ένα λειτουργικό κενό λίγο πριν την ολοκλήρωση του έργου.

Παράλληλα, ανέλαβα την ανάπτυξη του συστήματος αποστολής email επιβεβαίωσης ραντεβού (*email_service.py*), το οποίο αποτελούσε βασική λειτουργική απαίτηση για την άμεση ενημέρωση των πελατών. Η υλοποίηση χρειάστηκε περίπου τρεις ώρες καθαρής ανάπτυξης, καθώς και επιπλέον χρόνο για τη δημιουργία και παραμετροποίηση του λογαριασμού Gmail που χρησιμοποιήθηκε για τις δοκιμές και την παρουσίαση. Παρότι η προετοιμασία αυτή μπορεί να φαίνεται δευτερεύουσα, στην πράξη ήταν κρίσιμη για την αξιόπιστη επίδειξη της λειτουργίας.

Επιπλέον, στο *email_service.py* η αποστολή του email επιβεβαίωσης υλοποιήθηκε με χρήση *multithreading*, καθώς η επικοινωνία με τον SMTP server είναι blocking I/O λειτουργία. Η διαδικασία αποστολής απαιτεί άνοιγμα σύνδεσης δικτύου, χειραψία ασφαλείας (handshake), αυθεντικοποίηση (authentication) και αναμονή απάντησης από τον εξυπηρετητή, γεγονός που εισάγει αισθητή καθυστέρηση. Δεδομένου ότι το γραφικό περιβάλλον (GUI) της εφαρμογής βασίζεται στην Tkinter και εκτελείται σε ένα μόνο κύριο νήμα (UI thread), η εκτέλεση της αποστολής στο ίδιο νήμα θα προκαλούσε στιγμιαίο «πάγωμα» του παραθύρου. Για να αποφευχθεί αυτή η υποβάθμιση της

εμπειρίας χρήσης, η αποστολή εκτελείται ασύγχρονα σε ξεχωριστό background thread (δηλωμένο ως daemon, ώστε να τερματίζεται αυτόματα με το κλείσιμο της εφαρμογής). Με αυτόν τον τρόπο, το GUI παραμένει πλήρως ανταποκρίσιμο κατά την επιβεβαίωση του ραντεβού, ενώ η βάση δεδομένων ενημερώνεται ομαλά μετά την ολοκλήρωση της διαδικασίας.

Συνολικά, η τελική φάση ήταν η πιο απαιτητική περίοδος του project. Ήταν το σημείο όπου η θεωρητική κατανομή ρόλων έδωσε τη θέση της σε μια πιο πρακτική λογική ολοκλήρωσης: έπρεπε να κλείσουν εκκρεμότητες, να συμπληρωθούν λειτουργίες και να συνδεθούν όλα σε ένα ενιαίο αποτέλεσμα. Παρότι απαιτητική, αυτή η περίοδος μου έδειξε στην πράξη τι σημαίνει να αναλαμβάνεις ευθύνη σε μια ομαδική ανάπτυξη λογισμικού όταν η πίεση γίνεται πραγματική και το έργο πρέπει να παραδοθεί.

6. Συνολική αποτίμηση της διαδικασίας και συμπεράσματα

Σε προσωπικό επίπεδο, η συμμετοχή μου στο συγκεκριμένο project αποτέλεσε μία από τις πιο ουσιαστικές εμπειρίες που έχω ζήσει μέχρι σήμερα στον χώρο της ανάπτυξης λογισμικού. Από τον Φεβρουάριο μέχρι και την τελική παράδοση χρειάστηκε να κινηθώ ταυτόχρονα σε πολλαπλούς ρόλους: προγραμματιστής, τεχνικά υπεύθυνος, συντονιστής και, σε αρκετές περιπτώσεις, το άτομο που διατηρούσε τη συνολική εικόνα του έργου όταν η πίεση αυξανόταν. Αυτή η πολυδιάστατη συμμετοχή με βοήθησε να κατανοήσω βαθύτερα ότι η ανάπτυξη λογισμικού δεν είναι μόνο ζήτημα κώδικα, αλλά και ζήτημα οργάνωσης, συνεργασίας, συνέπειας και διαχείρισης απρόβλεπτων καταστάσεων.

Ανατρέχοντας στην πορεία του έργου, θεωρώ ότι η σημαντικότερη συμβολή μου ήταν η ενεργή παρουσία μου σε όλα τα κρίσιμα στάδια της ανάπτυξης. Από τον αρχικό αρχιτεκτονικό σχεδιασμό και τη διαμόρφωση του σκελετού της εφαρμογής, μέχρι την υλοποίηση της βάσης δεδομένων, την ανάπτυξη της διεπαφής χρήστη, την κάλυψη λειτουργικών κενών και την τελική ολοκλήρωση του συστήματος, προσπάθησα να λειτουργήσω με συνέπεια και υπευθυνότητα. Το τελικό αποτέλεσμα —μια λειτουργική, σταθερή και επεκτάσιμη εφαρμογή διαχείρισης ραντεβού— αντικατοπτρίζει τον χρόνο, την προσπάθεια και την επιμονή που απαιτήθηκαν σε όλη αυτή τη διαδρομή.

Πέρα όμως από το τεχνικό κομμάτι, το project αυτό λειτούργησε για μένα και ως μια πραγματική άσκηση ωριμότητας. Με έφερε αντιμέτωπο με καθυστερήσεις, ασυμβατότητες, ανάγκη για ανασχεδιασμό, πίεση χρόνου και λήψη άμεσων αποφάσεων. Σε κάθε μία από αυτές τις στιγμές έπρεπε να βρω λύσεις που να εξυπηρετούν όχι μόνο το δικό μου κομμάτι, αλλά και τη συνολική πορεία της ομάδας. Αυτό με βοήθησε να αναπτύξω δεξιότητες που δεν περιορίζονται στον προγραμματισμό, αλλά επεκτείνονται στη συνεργασία, στην επικοινωνία και στη διαχείριση ευθύνης.

Συνολικά, η εργασία αυτή δεν ήταν απλώς μια ακαδημαϊκή υποχρέωση. Ήταν μια ολοκληρωμένη εμπειρία ανάπτυξης λογισμικού, από την αρχική σύλληψη μέχρι την τελική παράδοση. Με βοήθησε να δω στην πράξη πώς συνδέονται οι τεχνικές αποφάσεις με την πραγματική λειτουργικότητα, πώς η αρχιτεκτονική επηρεάζει την καθημερινή χρήση και πώς η ομαδική συνεργασία μπορεί να μετατρέψει ένα θεωρητικό πλάνο σε ένα πραγματικό προϊόν. Θεωρώ ότι βγήκα από αυτή τη διαδικασία πιο ώριμος τεχνικά, πιο σίγουρος για τις δυνατότητές μου και με καθαρότερη εικόνα για το πώς θέλω να εξελιχθώ ως μηχανικός λογισμικού.

7. Παραδοχές και περιορισμοί της εφαρμογής

Στο σημείο αυτό συγκεντρώνω τις βασικές παραδοχές και τους περιορισμούς που αφορούν κυρίως τα τμήματα της εφαρμογής στα οποία εργάστηκα προσωπικά. Οι επιλογές αυτές συνδέονται άμεσα με το διαθέσιμο χρονικό πλαίσιο, τις τεχνικές απαιτήσεις της εργασίας και τη συνολική αρχιτεκτονική που διαμορφώθηκε στην έκδοση V1.0. Παράλληλα, αρκετοί από τους περιορισμούς αποτελούν συνειδητές αποφάσεις σχεδιασμού, με στόχο η εφαρμογή να παραμείνει σταθερή και λειτουργική στο πλαίσιο του μαθήματος, αφήνοντας όμως περιθώριο για μελλοντική επέκταση σε πιο απαιτητικά περιβάλλοντα.

1. Τοπική βάση δεδομένων και single-user λειτουργία

Η εφαρμογή βασίζεται σε τοπική βάση SQLite και έχει σχεδιαστεί για χρήση από έναν μόνο σταθμό εργασίας (PC/laptop). Για τις ανάγκες της εργασίας αυτή η επιλογή ήταν επαρκής, καθώς επέτρεψε γρήγορη ανάπτυξη και απλοποίηση της αρχιτεκτονικής. Ωστόσο, σε πραγματικό περιβάλλον με πολλούς χρήστες θα απαιτούνταν μετάβαση σε client-server μοντέλο (π.χ. PostgreSQL) και υλοποίηση μηχανισμών διαχείρισης ταυτόχρονων εγγραφών.

2. Ασφάλεια δεδομένων και GDPR

Παρότι η ταυτοποίηση των χρηστών υλοποιείται με hashing κωδικών, η τοπική βάση SQLite δεν είναι κρυπτογραφημένη στο επίπεδο του δίσκου. Σε παραγωγικό περιβάλλον όπου αποθηκεύονται προσωπικά δεδομένα πελατών, θα ήταν απαραίτητη η χρήση κρυπτογραφημένης βάσης (όπως SQLCipher) και επιπλέον μέτρα προστασίας για πλήρη συμμόρφωση με τους κανονισμούς GDPR.

3. Υπηρεσία αποστολής email

Η αποστολή email υλοποιείται μέσω SMTP και έχει δοκιμαστεί επιτυχώς με λογαριασμό Gmail (App Passwords). Για να μην επηρεάζεται η ανταπόκριση του γραφικού περιβάλλοντος, η αποστολή εκτελείται ασύγχρονα σε ξεχωριστό νήμα. Η λειτουργία απαιτεί ενεργή σύνδεση στο διαδίκτυο και σωστή παραμετροποίηση του λογαριασμού. Σε μελλοντική παραγωγική έκδοση θα ήταν απαραίτητος ένας persistent μηχανισμός ουράς εργασιών (task queue) με αυτόματες επαναπροσπάθειες (retries) σε περίπτωση αποτυχίας σύνδεσης.

4. Ανταπόκριση και περιορισμοί του γραφικού περιβάλλοντος

Λόγω των εγγενών περιορισμών της Tkinter στη δυναμική αναδιάταξη στοιχείων (responsive layout), έχει οριστεί σταθερό ελάχιστο μέγεθος παραθύρου (980×680). Η εφαρμογή προσαρμόζεται σε μεγαλύτερες αναλύσεις, αλλά η σμίκρυνση κάτω από αυτό το όριο έχει απενεργοποιηθεί σκόπιμα, ώστε να διασφαλίζεται ότι τα στοιχεία της διεπαφής δεν επικαλύπτονται και ότι η εμπειρία χρήσης παραμένει συνεπής.

5. Αντίγραφα ασφαλείας και αποθήκευση δεδομένων

Ο μηχανισμός αντιγράφων ασφαλείας δημιουργεί τοπικά backups στον φάκελο /backups, με αυτόματο καθαρισμό παλαιότερων αρχείων και διατήρηση του πιο πρόσφατου. Δεν έχει υλοποιηθεί συγχρονισμός με cloud υπηρεσίες ή απομακρυσμένη αποθήκευση, κάτι που θα ήταν απαραίτητο σε

πραγματικό επιχειρησιακό περιβάλλον για προστασία από απώλεια δεδομένων λόγω βλάβης του δίσκου.

6. Back-office λειτουργία και ευέλικτη διαχείριση διαθεσιμότητας

Μια βασική παραδοχή σχεδιασμού είναι ότι η εφαρμογή λειτουργεί ως εργαλείο εσωτερικής οργάνωσης της επιχείρησης και απευθύνεται σε εξουσιοδοτημένο προσωπικό, όπως γραμματεία ή διαχειριστή, και όχι στους ίδιους τους πελάτες μέσω αυτόνομης πλατφόρμας κρατήσεων. Στο πλαίσιο αυτό, το σύστημα έχει σχεδιαστεί ώστε να παρέχει αυξημένη ευελιξία στη διαχείριση της διαθεσιμότητας της επιχείρησης, υποστηρίζοντας τόσο το τυπικό εβδομαδιαίο πρόγραμμα λειτουργίας όσο και ειδικές εξαιρέσεις για συγκεκριμένες ημερομηνίες, όπως έκτακτες αργίες, διακοπές, τοπικές εορτές ή προσωρινές αλλαγές ωραρίου. Η καταχώρηση, η τροποποίηση και η ακύρωση των ραντεβού πραγματοποιείται από τον διαχειριστή κατόπιν επικοινωνίας με τον πελάτη, ενώ ο μηχανισμός ελέγχου διαθεσιμότητας προσαρμόζεται αυτόματα στις δηλωμένες εξαιρέσεις, αποτρέποντας κρατήσεις σε μη εργάσιμες ώρες.

7. Απουσία διασύνδεσης με εξωτερικά ημερολόγια

Η διαχείριση της διαθεσιμότητας και των κρατήσεων πραγματοποιείται αποκλειστικά μέσα στο κλειστό σύστημα της εφαρμογής και στηρίζεται στα δεδομένα της τοπικής βάσης. Δεν υποστηρίζεται αυτόματος συγχρονισμός με εξωτερικές cloud υπηρεσίες ημερολογίου, όπως Google Calendar ή Outlook Calendar. Ως αποτέλεσμα, η ορθότητα της διαθεσιμότητας εξαρτάται αποκλειστικά από τη συνέπεια ενημέρωσης των εγγραφών εντός της εφαρμογής, γεγονός που συνιστά έναν σαφή λειτουργικό περιορισμό της έκδοσης V1.0.

8. Συμβατότητα packaged εκτέλεσης

Παρότι ο πηγαίος κώδικας της εφαρμογής σε Python παραμένει θεωρητικά cross-platform, το τελικό εκτελέσιμο αρχείο που πακεταρίστηκε μέσω PyInstaller έχει σχεδιαστεί και δοκιμαστεί αποκλειστικά σε περιβάλλον Windows. Για χρήση σε άλλα λειτουργικά συστήματα, όπως macOS ή Linux, θα απαιτούνταν είτε εκτέλεση του έργου μέσω διερμηνέα Python είτε δημιουργία ξεχωριστού build προσαρμοσμένου στο εκάστοτε περιβάλλον. Ο περιορισμός αυτός αφορά κυρίως τη διανεμόμενη μορφή της εφαρμογής και όχι τη βασική λογική του πηγαίου κώδικα.

Οι παραπάνω παραδοχές και περιορισμοί δεν πρέπει να εκληφθούν ως αδυναμίες της εφαρμογής, αλλά ως συνειδητές επιλογές που διαμορφώθηκαν τόσο από τις πραγματικές ανάγκες μιας πρώτης λειτουργικής υλοποίησης όσο και από το ίδιο το πλαίσιο του θέματος που δόθηκε στο μάθημα. Με άλλα λόγια, αντανakλούν όχι μόνο μια ρεαλιστική προσέγγιση σχεδιασμού, αλλά και την προσπάθεια να υλοποιηθεί με συνέπεια ένα σύστημα που να ανταποκρίνεται ουσιαστικά στις απαιτήσεις της συγκεκριμένης ακαδημαϊκής εργασίας. Παράλληλα, αναδεικνύουν με σαφήνεια τα σημεία στα οποία μπορεί να στηριχθεί μια μελλοντική αναβάθμιση, εφόσον το RandeBoo εξελιχθεί σε ένα πιο ολοκληρωμένο σύστημα διαχείρισης επιχειρησιακών λειτουργιών μιας επιχείρησης.

8. Επίλογος

Κοιτώντας πίσω σε όλη αυτή τη διαδρομή, συνειδητοποιώ ότι το μεγαλύτερο κέρδος μου δεν ήταν μόνο οι τεχνικές γνώσεις, αλλά η κατανόηση του τρόπου με τον οποίο γεννιέται και εξελίσσεται ένα σύστημα. Το project αυτό μου έδειξε ότι η ανάπτυξη λογισμικού δεν είναι μια ευθεία γραμμή, αλλά μια διαδικασία συνεχών επιλογών, αναθεωρήσεων και μικρών αποφάσεων που τελικά διαμορφώνουν κάτι μεγαλύτερο από το άθροισμα των μερών του.

Αυτό που ανακάλυψα για τον εαυτό μου είναι ότι με ελκύει περισσότερο η αρχιτεκτονική σκέψη παρά η απομονωμένη γραφή κώδικα. Με ενδιαφέρει το πώς “κάθεται” ένα σύστημα, πώς οργανώνεται, πώς αναπνέει, πώς μπορεί να επεκταθεί χωρίς να καταρρεύσει. Με ενδιαφέρει η συνοχή, η καθαρότητα και η αίσθηση ότι κάθε κομμάτι έχει θέση και σκοπό. Αυτή η ορχηστρωτική προσέγγιση —το να βλέπεις το σύνολο και όχι μόνο το επιμέρους— είναι κάτι που θέλω να καλλιεργήσω συνειδητά στη συνέχεια της πορείας μου.

Σε μια περίοδο όπου τα εργαλεία τεχνητής νοημοσύνης ενσωματώνονται όλο και περισσότερο στην καθημερινότητα της ανάπτυξης λογισμικού, θεωρώ ότι η ουσιαστική πρόκληση για έναν μηχανικό δεν είναι απλώς να τα χρησιμοποιεί, αλλά να τα αξιοποιεί με κρίση, μέτρο και υπευθυνότητα. Η εμπειρία μου μέσα από το συγκεκριμένο project επιβεβαίωσε ότι τα εργαλεία αυτά δεν μπορούν —και δεν πρέπει— να υποκαθιστούν τη διαδικασία ανάπτυξης, καθώς ο αυτόματος παραγόμενος κώδικας συχνά συνοδεύεται από σφάλματα, παραλείψεις ή λογικές ασυνέπειες που δεν είναι πάντοτε άμεσα ορατές κατά τη φάση της υλοποίησης.

Για τον λόγο αυτό, αντιμετώπισα την τεχνητή νοημοσύνη όχι ως μηχανισμό παραγωγής λύσεων, αλλά ως εργαλείο σκέψης και μάθησης. Με βοήθησε να εξετάσω εναλλακτικές προσεγγίσεις, να κατανοήσω ταχύτερα νέες έννοιες και να οργανώσω πιο καθαρά τη λογική μου, χωρίς όμως να μεταφέρεται σε αυτήν η ευθύνη της τελικής απόφασης. Η ποιότητα, η αξιοπιστία και η συνοχή ενός συστήματος παραμένουν τελικά ευθύνη του ίδιου του μηχανικού.

Μέσα από αυτή τη διαδικασία συνειδητοποίησα ότι η σωστή αξιοποίηση των σύγχρονων εργαλείων δεν βρίσκεται στην άκριτη χρήση τους, αλλά στην ικανότητα να λειτουργούν ως ενίσχυση της ανθρώπινης κρίσης και όχι ως υποκατάστατό της. Αυτή είναι και η στάση που θέλω να διατηρήσω στη συνέχεια της πορείας μου ως μηχανικός λογισμικού.

Το project αυτό ήταν ένα πρώτο βήμα προς αυτή την κατεύθυνση. Με έφερε αντιμέτωπο με πραγματικές δυσκολίες, με περιορισμούς, με πίεση, αλλά και με τη χαρά του να βλέπεις κάτι να παίρνει μορφή επειδή το οργάνωσες σωστά και το υποστήριξες μέχρι το τέλος. Και μέσα από αυτή τη διαδικασία κατάλαβα λίγο καλύτερα όχι μόνο το πώς θέλω να δουλεύω, αλλά και το πώς θέλω να εξελιχθώ ως μηχανικός λογισμικού: με σκέψη καθαρή, με αρχιτεκτονική συνείδηση, με ικανότητα να αξιοποιώ σύγχρονα εργαλεία με υπευθυνότητα και με διάθεση να χτίζω συστήματα που έχουν διάρκεια και νόημα.

9. Πηγές και βιβλιογραφία υλοποίησης

Στην ενότητα αυτή παραθέτω τις βασικές πηγές, τα εργαλεία και το εκπαιδευτικό υλικό που αξιοποίησα κατά την ανάπτυξη των επιμέρους τμημάτων της εφαρμογής. Οι πηγές αυτές συνέβαλαν τόσο στην κατανόηση των τεχνολογιών που χρησιμοποιήθηκαν όσο και στην πρακτική αντιμετώπιση τεχνικών ζητημάτων που προέκυψαν κατά τη διάρκεια της υλοποίησης.

9.1 database.py (Διαχείριση βάσης δεδομένων – SQLite3)

Python Software Foundation. 2026. *sqlite3 — DB-API 2.0 interface for SQLite databases*. Available at: <https://docs.python.org/3/library/sqlite3.html>

SQLite Tutorial. n.d. *SQLite Tutorial – An Easy Way to Master SQLite Fast*. Available at: <https://www.sqlitetutorial.net/>

Schafer, C. 2017. *Python SQLite Tutorial: Complete Overview* [online video]. Available at: <https://www.youtube.com/watch?v=pd-0G0MigUA>

Farrell, D. n.d. *Data Management With Python, SQLite, and SQLAlchemy*. Available at: <https://realpython.com/python-sqlite-sqlalchemy/>

9.1.1 seed_db.py (Δημιουργία mockup data – Faker)

Faker Documentation. 2026. *Faker Documentation — Release 40.21.0*. Available at: <https://faker.readthedocs.io/en/master/>

de Uña, D. (joke2k). 2026. *Faker — A Python package that generates fake data for you*. Available at: <https://github.com/joke2k/faker/>

9.2 backup.py (Αντίγραφα ασφαλείας – File Management)

Python Software Foundation. n.d. *shutil — High-level file operations*. Available at: <https://docs.python.org/3/library/shutil.html>

Python Software Foundation. n.d. *shutil.copy2 — copy2(src, dst, *, follow_symlinks=True)*. Available at: <https://docs.python.org/3/library/shutil.html#shutil.copy2>

Python Software Foundation. 2026. *os.path — Common pathname manipulations*. Available at: <https://docs.python.org/3/library/os.path.html>

Stack Overflow. n.d. *How do I copy a file?* Available at: <https://stackoverflow.com/questions/123198/how-do-i-copy-a-file>

Ndlovu, V. 2018. *Working With Files in Python*. Available at: <https://realpython.com/working-with-files-in-python/>

9.3 email_service.py (Αποστολή email – SMTP)

Python Software Foundation. 2026. *smtplib — SMTP protocol client*. Available at: <https://docs.python.org/3/library/smtplib.html>

Python Software Foundation. 2026. *email — An email and MIME handling package*. Available at: <https://docs.python.org/3/library/email.html>

Schafer, C. n.d. *Python Tutorial: How to Send Emails* [online video]. Available at: <https://www.youtube.com/watch?v=JRCJ6RtE3xU>

Real Python. n.d. *Sending Emails With Python*. Available at:
<https://realpython.com/python-send-email/>

Python Software Foundation. 2026. *threading — Thread-based parallelism*. Available at:
<https://docs.python.org/3/library/threading.html>

Stack Overflow. n.d. *Tkinter: How to use threads to prevent the main event loop from freezing*. Available at: <https://stackoverflow.com/questions/16745507/tkinter-how-to-use-threads-to-preventing-main-event-loop-from-freezing>

Python Software Foundation. 2026. *email.mime: Creating email and MIME objects from scratch*. Available at: <https://docs.python.org/3/library/email.mime.html>

Google Support. n.d. *Sign in with App Passwords (SMTP/IMAP/POP)*. Available at:
<https://support.google.com/accounts/answer/>

9.4 GUI Modules (Tkinter & tkcalendar)

gui_main.py, gui_settings.py, gui_employees.py, gui_quick_booking.py

Python Software Foundation. n.d. *tkinter — Python interface to Tcl/Tk*. Available at:
<https://docs.python.org/3/library/tkinter.html>

Elder, J. n.d. *Tkinter.com*. Available at: <https://tkinter.com/>

Python Guides. n.d. *Python Tkinter Treeview*. Available at:
<https://pythonguides.com/python-tkinter-treeview/>

Codemy.com. n.d. *Modern Tkinter Design* [online playlist]. Available at:
<https://www.youtube.com/playlist?list=PLCC34OHNCotoC6GglhF3ncJ5rLwQrLGnV>

Stack Overflow. n.d. *Switch between two frames in Tkinter*. Available at:
<https://stackoverflow.com/questions/7546050/switch-between-two-frames-in-tkinter>

Python Package Index. n.d. *tkcalendar*. Available at: <https://pypi.org/project/tkcalendar/>

Python GUIs. n.d. *Python GUIs – Tutorials, Examples & Guides for Building GUIs in Python*. Available at: <https://www.pythonguis.com/>

9.5 main.py (Entry Point & App Structure)

Real Python. n.d. *What Does if name == "main" Do in Python?*. Available at:
<https://realpython.com/if-name-main-python/>

Real Python. n.d. *Python Application Layouts: A Reference*. Available at:
<https://realpython.com/python-application-layouts/>

9.6 Γενικά εργαλεία και οδηγοί

Stack Overflow. n.d. *Python questions tagged "python"*. Available at:
<https://stackoverflow.com/questions/tagged/python>

Αγγελιδάκης, Ν.Α. 2015. *Εισαγωγή στον προγραμματισμό με την Python*.

Swaroop, C.H. 2013. *A Byte of Python*.

Μαγκούτης, Κ. & Νικολάου, Χ. 2015. *Εισαγωγή στον αντικειμενοστραφή προγραμματισμό με Python*.

9.7 Προχωρημένες τεχνικές και υποστηρικτικό υλικό

Ψούνης, Γ. 2026. *Μίνι σειρές στην Python*. Available at:
https://github.com/psounis/python_advanced

9.8 PyInstaller (Packaging & Executable Build)

PyInstaller Development Team. 2026. *Run-time Information*. Available at:
<https://pyinstaller.org/en/stable/runtime-information.html>

PyInstaller Development Team. 2026. *Understanding PyInstaller Hooks*. Available at:
<https://pyinstaller.org/en/stable/hooks.html>

PyInstaller Development Team. 2026. *Using Spec Files*. Available at:
<https://pyinstaller.org/en/stable/spec-files.html>

Stack Overflow. n.d. *Determining application path in a Python EXE generated by PyInstaller*. Available at: <https://stackoverflow.com/questions/404744/determining-application-path-in-a-python-exe-generated-by-pyinstaller>